

Sistema de Gestión de Gimnasio con Spring Boot – Guía de Estudio

Análisis estructurado del código real del proyecto Spring Boot para estudiar MVC, REST, JPA, DTO y POO de forma progresiva.

Proyecto analizado	com.gym:gym-system
Tecnologías	Spring Boot 4.0.6, Java 17+, Spring Web MVC, Spring Data JPA, H2
Persistencia activa	H2 en memoria (jdbc:h2:mem:testdb)
Validación realizada	Compilación y prueba contextLoads ejecutadas con éxito el 03/05/2026

Nota de análisis

La guía se basa en el estado actual del código. El flujo CREATE, READ y UPDATE está operativo; DELETE aparece en el diseño del servicio y en el README, pero todavía no está expuesto en el controlador y su implementación actual lanza UnsupportedOperationException.

1. Introducción

Este sistema tiene como objetivo gestionar de forma básica a los miembros de un gimnasio. El problema que resuelve es sencillo pero muy útil para aprender backend: registrar personas, consultar el listado, buscar un miembro por su identificador y actualizar los datos de su membresía desde una API REST.

Desde el punto de vista académico, el proyecto también sirve como laboratorio de conceptos clave de Spring Boot. En un mismo código aparecen la arquitectura MVC, la separación por capas, la persistencia con JPA/Hibernate, el uso de DTO para desacoplar la API de la entidad y varios ejemplos de Programación Orientada a Objetos como abstracción, herencia, encapsulación y polimorfismo.

El dominio principal del sistema es la gestión de miembros. La entidad central es Member, y a su alrededor se organizan las capas controller, service, repository y dto. Esa organización hace que el proyecto sea pequeño, pero lo suficientemente completo para estudiar cómo se construye una API profesional desde cero.

2. Arquitectura del Proyecto (MVC)

La aplicación sigue una variante de MVC adaptada a una API REST. En lugar de vistas HTML, el sistema devuelve JSON. La idea central sigue siendo la misma: cada capa tiene una responsabilidad clara y evita mezclar lógica de transporte, negocio y acceso a datos.

Controller	Recibe solicitudes HTTP, extrae parámetros y delega al servicio.
Service	Aplica la lógica del caso de uso y coordina conversión DTO ↔ entidad.
Repository	Habla con la base de datos a través de JpaRepository.
Model	Representa los datos del dominio, por ejemplo la entidad Member.

2.1 Ejemplo de interacción entre capas

Cuando un cliente hace un POST a /api/members, el flujo real del proyecto es el siguiente: el controlador recibe el JSON en un MemberRequestDTO, el servicio lo convierte a Member mediante MemberMapper, el repositorio lo guarda con repository.save(member) y finalmente el servicio devuelve un MemberResponseDTO al controlador para responder en JSON.

```
@PostMapping
public MemberResponseDTO create(@RequestBody MemberRequestDTO dto) {
    return service.save(dto);
}
```

El controlador casi no contiene lógica de negocio. Esa es una buena práctica porque mantiene la API limpia y facilita las pruebas. La inteligencia principal queda en MemberServiceImpl, mientras que MemberRepository aprovecha el CRUD automático que ya ofrece Spring Data JPA.

```
public interface MemberRepository extends JpaRepository<Member, Long> {
}
```

3. Entidad Principal (Member)

La clase Member representa la tabla principal del sistema. Es la estructura que Hibernate usa para persistir los datos en H2. A continuación se muestra la clase completa tal como aparece en el proyecto:

```
package com.gym.gym_system.model;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

/**
 * @Entity indica que esta clase es una tabla en la BD
 */
@Entity
public class Member {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;
    private String name;
    private int age;
```

```
private String membershipType;

// Constructor vacío (requerido por JPA)
public Member() {
}

// Constructor completo
public Member(String name, int age, String membershipType) {
    this.name = name;
    this.age = age;
    this.membershipType = membershipType;
}

// Encapsulación → getters y setters
public Long getId() {
    return id;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

// Sobrecarga de métodos (overloading)
public void setName(String name, String lastName) {
    this.name = name + " " + lastName;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

public String getMembershipType() {
    return membershipType;
}

public void setMembershipType(String membershipType) {
    this.membershipType = membershipType;
}
}
```

3.1 Explicación de las anotaciones y de la encapsulación

- `@Entity` indica que la clase debe mapearse como tabla de base de datos.
- `@Id` marca el atributo `id` como clave primaria.
- `@GeneratedValue(strategy = GenerationType.IDENTITY)` delega a la base de datos la generación automática del identificador.

- Los atributos son privados, así que el acceso se hace por getters y setters. Esa es la base de la encapsulación.
- El constructor vacío es obligatorio para JPA, porque Hibernate necesita crear objetos de la entidad internamente.

Observa además que `membershipType` y `age` viven en la entidad, pero la respuesta pública de la API no expone `age`. Esa diferencia existe porque la API devuelve un DTO distinto a la entidad persistida.

4. Programación Orientada a Objetos aplicada

El proyecto incorpora varios conceptos de POO. Algunos participan directamente en el flujo REST, y otros aparecen como ejemplos didácticos dentro del código para reforzar teoría orientada a objetos.

4.1 Encapsulación

La encapsulación se ve claramente en `Member`: `id`, `name`, `age` y `membershipType` son atributos privados. Nadie desde fuera modifica esos datos directamente; siempre se hace mediante getters y setters. Esto protege el estado interno y permite agregar reglas en el futuro sin romper al resto de la aplicación.

4.2 Herencia (`AbstractPerson` → `Person`)

La herencia aparece en la relación entre `AbstractPerson` y `Person`. `AbstractPerson` define datos y comportamiento base, y `Person` reutiliza esa estructura sin duplicar código.

```
public abstract class AbstractPerson {

    protected String name;
    protected int age;

    public abstract String getRole();

    public String basicInfo() {
        return name + " - " + age;
    }
}

public class Person extends AbstractPerson {

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    @Override
    public String getRole() {
        return "Basic Person";
    }
}
```

Aquí Person hereda los atributos protegidos name y age, además del método concreto basicInfo(). También está obligada a implementar getRole(), porque ese método es abstracto en la clase padre.

4.3 Polimorfismo (PremiumMember)

PremiumMember extiende Member y añade un nuevo atributo llamado benefits. Esto demuestra polimorfismo porque un objeto PremiumMember también puede ser tratado como Member, pero con comportamiento o datos adicionales.

```
public class PremiumMember extends Member {  
    private String benefits;  
  
    public PremiumMember(String name, int age, String membershipType, String benefits)  
    {  
        super(name, age, membershipType);  
        this.benefits = benefits;  
    }  
  
    public String getBenefits() {  
        return benefits;  
    }  
}
```

En un sistema más grande, esto permitiría manejar diferentes tipos de miembros compartiendo una base común. En este proyecto, la clase cumple principalmente una función pedagógica.

4.4 Abstracción

La abstracción se materializa en AbstractPerson. Al declarar una clase abstracta, el proyecto expresa una idea general de persona sin instanciarla directamente. La clase describe qué cosas toda persona debe tener y qué comportamiento debe completar cada subclase.

5. Interfaces

Las interfaces sirven para definir contratos. Es decir: especifican qué debe poder hacer una clase, sin decidir todavía cómo lo hará. Esta separación es clave para desacoplar la lógica y poder cambiar implementaciones sin alterar el resto del sistema.

5.1 MemberService

```
public interface MemberService {  
  
    MemberResponseDTO save(MemberRequestDTO dto);  
    List<MemberResponseDTO> findAll();  
    MemberResponseDTO findById(Long id);  
    MemberResponseDTO update(Long id, MemberRequestDTO dto);  
    void delete(Long id);  
}
```

MemberService define el contrato principal del caso de uso de miembros. MemberServiceImpl es la clase que cumple ese contrato. Gracias a esa separación, el controlador depende de la abstracción MemberService y no de una clase concreta.

5.2 PaymentService

```
public interface PaymentService {  
  
    void processPayment(double amount);  
}
```

PaymentService todavía no tiene implementación en el proyecto, pero es útil para estudiar interfaces. El contrato dice que cualquier servicio de pagos debe saber procesar un monto. Más adelante podría existir una implementación con tarjeta, efectivo o pasarela externa.

6. Sobrecarga de Métodos

La sobrecarga ocurre cuando una clase tiene varios métodos con el mismo nombre, pero distinta lista de parámetros. En Member esto se observa con setName().

```
public void setName(String name) {  
    this.name = name;  
}  
  
public void setName(String name, String lastName) {  
    this.name = name + " " + lastName;  
}
```

La primera versión asigna solo el nombre; la segunda construye el nombre completo uniendo nombre y apellido. El compilador decide cuál ejecutar según la firma que se use al invocarlo.

7. API REST (CRUD)

La base URL del recurso es /api/members. La API trabaja con JSON y actualmente expone operaciones de alta, consulta y actualización. El borrado forma parte del diseño, pero todavía no está disponible como endpoint funcional.

Método	Endpoint	Estado en el proyecto
POST	/api/members	Implementado
GET	/api/members	Implementado
GET	/api/members/{id}	Implementado
PUT	/api/members/{id}	Implementado
DELETE	/api/members/{id}	Previsto, pero aún no implementado

7.1 POST /api/members

Crea un nuevo miembro. El cliente envía un JSON compatible con MemberRequestDTO.

```
Request  
POST /api/members  
Content-Type: application/json  
  
{
```

```
"name": "Arturo",
"age": 23,
"membershipType": "Mensual"
}
```

```
Response
{
  "id": 1,
  "name": "Arturo",
  "membershipType": "Mensual"
}
```

La respuesta no devuelve age porque MemberResponseDTO solo expone id, name y membershipType. Esto muestra claramente la diferencia entre el DTO de entrada y el DTO de salida.

7.2 GET /api/members

Devuelve todos los miembros registrados en forma de arreglo JSON.

```
Response
[
  {
    "id": 1,
    "name": "Arturo Mora",
    "membershipType": "Premium"
  },
  {
    "id": 2,
    "name": "Laura Torres",
    "membershipType": "Premium"
  }
]
```

7.3 GET /api/members/{id}

Busca un miembro específico por su id.

```
Request
GET /api/members/1
```

```
Response
{
  "id": 1,
  "name": "Arturo Mora",
  "membershipType": "Premium"
}
```

En la implementación actual, si el id no existe el servicio devuelve null. Como mejora futura, convendría responder con HTTP 404 Not Found y un mensaje controlado.

7.4 PUT /api/members/{id}

Actualiza los datos de un miembro existente.

```
Request
```

```
PUT /api/members/1
Content-Type: application/json
```

```
{
  "name": "Arturo Mora",
  "age": 24,
  "membershipType": "Premium"
}
```

Response

```
{
  "id": 1,
  "name": "Arturo Mora",
  "membershipType": "Premium"
}
```

Internamente, `MemberServiceImpl` primero busca el registro con `repository.findById(id)`, actualiza los campos y luego vuelve a guardar con `repository.save(existing)`.

7.5 DELETE /api/members/{id}

Este endpoint forma parte del CRUD esperado y también aparece en el README, pero aún no existe en `MemberController`. Además, el método `delete(Long id)` de `MemberServiceImpl` lanza `UnsupportedOperationException`. Por eso, al probar DELETE en el estado actual del proyecto, el servidor responde 405 Method Not Allowed.

Estado esperado en un CRUD completo
DELETE /api/members/1

Respuesta sugerida:
204 No Content

Observación práctica

Si implementas este endpoint más adelante, deberías agregar `@DeleteMapping("/{id}")` en `MemberController` y `repository.deleteById(id)` en `MemberServiceImpl`.

8. DTO (Data Transfer Object)

Un DTO es un objeto diseñado para transportar datos entre capas o entre la API y el cliente. Su finalidad es no exponer directamente la entidad de base de datos cuando eso no conviene.

- `MemberRequestDTO` representa los datos que entran desde POST y PUT: `name`, `age` y `membershipType`.
- `MemberResponseDTO` representa los datos que salen hacia el cliente: `id`, `name` y `membershipType`.
- Separar request y response hace posible controlar mejor qué campos entran y qué campos salen.

En este proyecto, el ejemplo práctico es muy claro: el cliente sí envía `age` al crear o actualizar, pero la API no devuelve `age` en la respuesta. Esa decisión es posible gracias al uso de DTO diferentes.

9. Mapper

El mapper centraliza la conversión entre DTO y entidad. Su función es evitar que el controlador o el servicio repitan manualmente la misma transformación una y otra vez.

```
public class MemberMapper {

    public static Member toEntity(MemberRequestDTO dto) {
        return new Member(
            dto.getName(),
            dto.getAge(),
            dto.getMembershipType()
        );
    }

    public static MemberResponseDTO toDTO(Member member) {
        return new MemberResponseDTO(
            member.getId(),
            member.getName(),
            member.getMembershipType()
        );
    }
}
```

La transformación ocurre así: cuando llega un `MemberRequestDTO`, `MemberMapper.toEntity(dto)` construye un `Member`. Después de guardar en la base de datos, `MemberMapper.toDTO(member)` genera el `MemberResponseDTO` final. En otras palabras, el flujo es `DTO → Entity → DTO`.

10. Base de Datos (H2)

La configuración activa del proyecto usa H2 en memoria. Esto es ideal para aprendizaje y pruebas rápidas porque no exige instalar un motor externo. Cada vez que la aplicación se reinicia, la base vuelve a empezar desde cero.

```
spring.application.name=gym-system
spring.datasource.url=jdbc:h2:mem:testdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=

spring.jpa.database-platform=org.hibernate.dialect.H2Dialect
spring.h2.console.enabled=true
spring.jpa.hibernate.ddl-auto=update
```

- `jdbc:h2:mem:testdb` crea una base de datos temporal en memoria.
- `org.h2.Driver` es el driver JDBC que permite conectarse a H2.
- `spring.h2.console.enabled=true` habilita la consola web de H2.
- `spring.jpa.hibernate.ddl-auto=update` permite que Hibernate ajuste la estructura según las entidades.

Aunque el `pom.xml` también incluye `mysql-connector-j`, la aplicación no está usando MySQL en este momento. La configuración efectiva está completamente orientada a H2.

11. Flujo completo del sistema

El flujo completo puede entenderse como una cadena de responsabilidad donde cada capa transforma o transporta información.

```
Cliente HTTP
↓
MemberController
↓
MemberServiceImpl
↓
MemberMapper
↓
MemberRepository
↓
H2 / Hibernate
↓
MemberResponseDTO
↓
Respuesta JSON
```

1. El cliente envía una petición HTTP con JSON.
2. El Controller recibe la petición y la convierte en un DTO de entrada.
3. El Service aplica la lógica de negocio y decide qué hacer.
4. El Mapper convierte el DTO a entidad cuando hace falta persistir.
5. El Repository usa JPA/Hibernate para acceder a la base de datos.
6. La base H2 guarda o recupera la información.
7. El Service vuelve a mapear la entidad hacia un DTO de salida.
8. El Controller devuelve la respuesta JSON al cliente.

12. Pruebas con Postman

Para probar la API en Postman, primero inicia la aplicación con `mvnw spring-boot:run`. Después crea una petición nueva y apunta a `http://localhost:8080/api/members`.

9. Selecciona el método POST.
10. En la pestaña Headers agrega Content-Type: application/json.
11. En Body elige raw y formato JSON.
12. Pega un cuerpo como el siguiente y presiona Send.

```
{
  "name": "Mariana",
  "age": 27,
  "membershipType": "Mensual"
}
```

Si todo está bien, recibirás una respuesta JSON con el id generado. Después puedes repetir el proceso con GET, GET por id y PUT. Para DELETE, recuerda que el proyecto todavía no lo soporta en su estado actual.

13. Problemas comunes y soluciones

13.1 Error 400 (Bad Request)

En este proyecto, el error 400 suele aparecer cuando el JSON está mal formado o cuando un campo llega con un tipo incompatible. Ejemplos típicos: comillas mal cerradas, comas extra o enviar age como texto en lugar de número.

Ejemplo de respuesta de error

```
{
  "status": 400,
  "error": "Bad Request",
  "path": "/api/members/2"
}
```

Además, como los DTO actuales no usan anotaciones de validación como `@NotBlank` o `@Min`, no verás mensajes de validación tan precisos. Eso es una mejora natural para el siguiente nivel del proyecto.

13.2 Puerto 8080 ocupado

Si Tomcat no puede iniciar porque el puerto 8080 ya está en uso, puedes cerrar el proceso que lo ocupa o cambiar temporalmente el puerto agregando `server.port=8081` en `application.properties`.

13.3 Problemas con JSON

Asegúrate de usar nombres de propiedades correctos: `name`, `age` y `membershipType`. También usa comillas dobles, no simples, y recuerda que un objeto JSON no lleva coma después del último campo.

13.4 Driver H2 faltante

Si la aplicación no reconoce `org.h2.Driver`, revisa que la dependencia de H2 esté presente en `pom.xml` y actualiza el proyecto Maven. En este repositorio la dependencia runtime de H2 ya existe, así que normalmente el problema se resuelve forzando la descarga de dependencias o recargando el proyecto en el IDE.

13.5 DELETE devuelve 405

Este no es un error de Postman: refleja el estado actual del código. El endpoint DELETE todavía no está mapeado en `MemberController`. La solución es implementarlo de forma completa en `controller` y `service`.

14. Conclusión

Al estudiar este proyecto se adquieren fundamentos muy valiosos de backend con Spring Boot: organización por capas, diseño MVC para APIs REST, persistencia con JPA/Hibernate, uso de DTO, mapeo entre objetos y aplicación práctica de conceptos de Programación Orientada a Objetos.

El siguiente paso natural consiste en fortalecer el sistema con validaciones, manejo formal de errores, relaciones entre entidades, seguridad con Spring Security, una base de datos real como MySQL o PostgreSQL y la implementación completa del endpoint DELETE junto con pruebas más profundas que el actual `contextLoads()`.

En resumen, este software ya funciona como una base didáctica sólida para aprender cómo se construye una API profesional, y al mismo tiempo deja visibles varias mejoras reales que permiten seguir creciendo paso a paso.

14.1 Próximos pasos sugeridos

- Agregar validaciones con Bean Validation, por ejemplo `@NotBlank`, `@Min` y `@Valid`.
- Responder errores de forma profesional con excepciones personalizadas y `@ControllerAdvice`.
- Completar el CRUD implementando DELETE y devolviendo 404 cuando un id no exista.
- Modelar nuevas entidades como pagos, planes o entrenadores, junto con sus relaciones JPA.
- Migrar de H2 a MySQL o PostgreSQL y añadir seguridad con Spring Security o JWT.